

Natural Language Processing Pipeline and Text Processing

Muhammad Yazid Supriadi, S.Kom, M.Kom
STT Terpadu Nurul Fikri

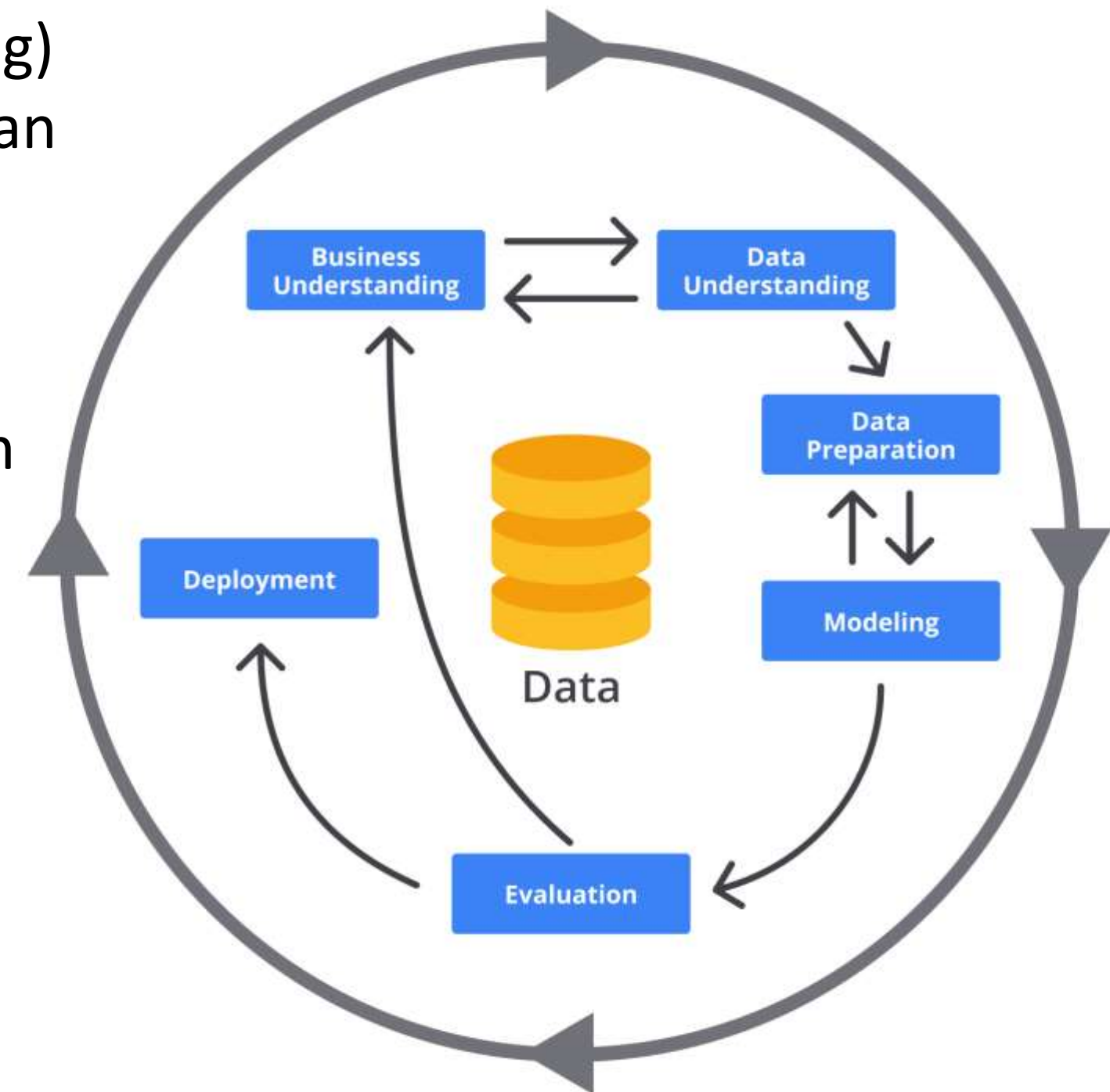
Cross Industry Standard Process for Data Mining (CRISP-DM)

CRISP-DM (Cross Industry Standard Process for Data Mining) adalah metodologi standar internasional untuk menjalankan proyek data mining dan data science secara sistematis dan terstruktur.

Metodologi ini dikembangkan tahun 1996 oleh konsorsium yang dipimpin oleh IBM bersama Daimler-Benz dan NCR Corporation, lalu dipublikasikan secara resmi pada 1999.

CRISP-DM bersifat:

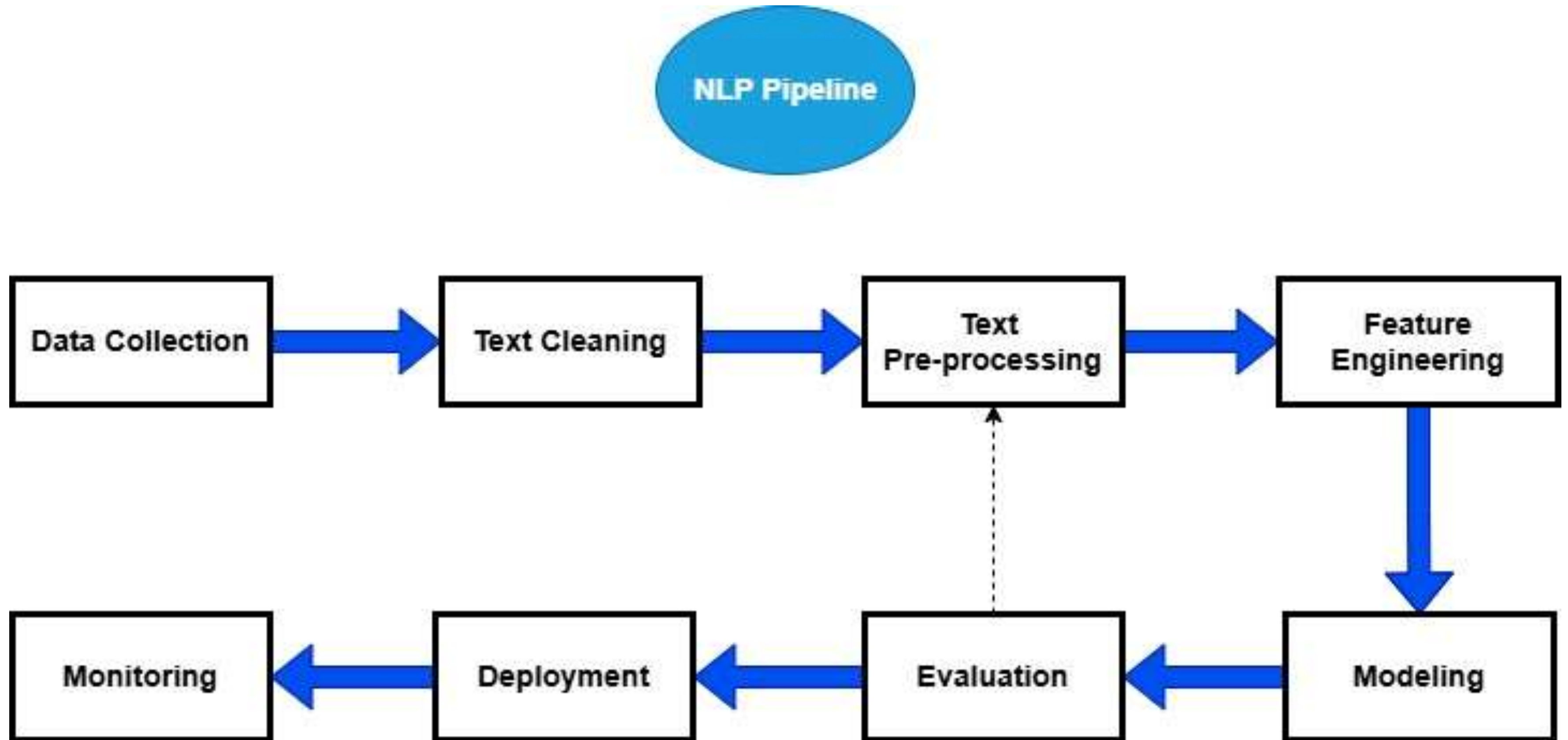
- Iteratif (bisa kembali ke tahap sebelumnya)
- Fleksibel (bisa digunakan di berbagai industri)
- Tidak tergantung tool tertentu
- Cocok untuk proyek Machine Learning & NLP



Cross Industry Standard Process for Data Mining (CRISP-DM)

Business Understanding	Data Understanding	Data Preparation	Modeling	Evaluation	Deployment
1. Determine Business Objectives	1. Collect Initial Data	1. Select Data	1. Select Modeling Techniques	1. Evaluate Results	1. Plan Deployment
2. Assess Situation	2. Describe Data	2. Clean Data	2. Generate Test Design	2. Review Process	2. Plan Monitoring & Maintenance
3. Determine Project Goals	3. Explore Data	3. Construct Data	3. Build Model	3. Determine Next Steps	3. Produce Final Report
4. Produce Project Plan	4. Verify Data Quality	4. Integrate Data 5. Format Data	4. Assess Model		4. Review Project

NLP Pipeline



Data Collection dalam NLP merupakan tahap pengumpulan korpus teks yang representatif terhadap domain masalah.

Dalam penelitian NLP, data collection bertujuan untuk:

- Mewakili distribusi bahasa sebenarnya
- Mendukung generalisasi model
- Mengurangi bias sampling
- Menyediakan data untuk training, validation, dan testing

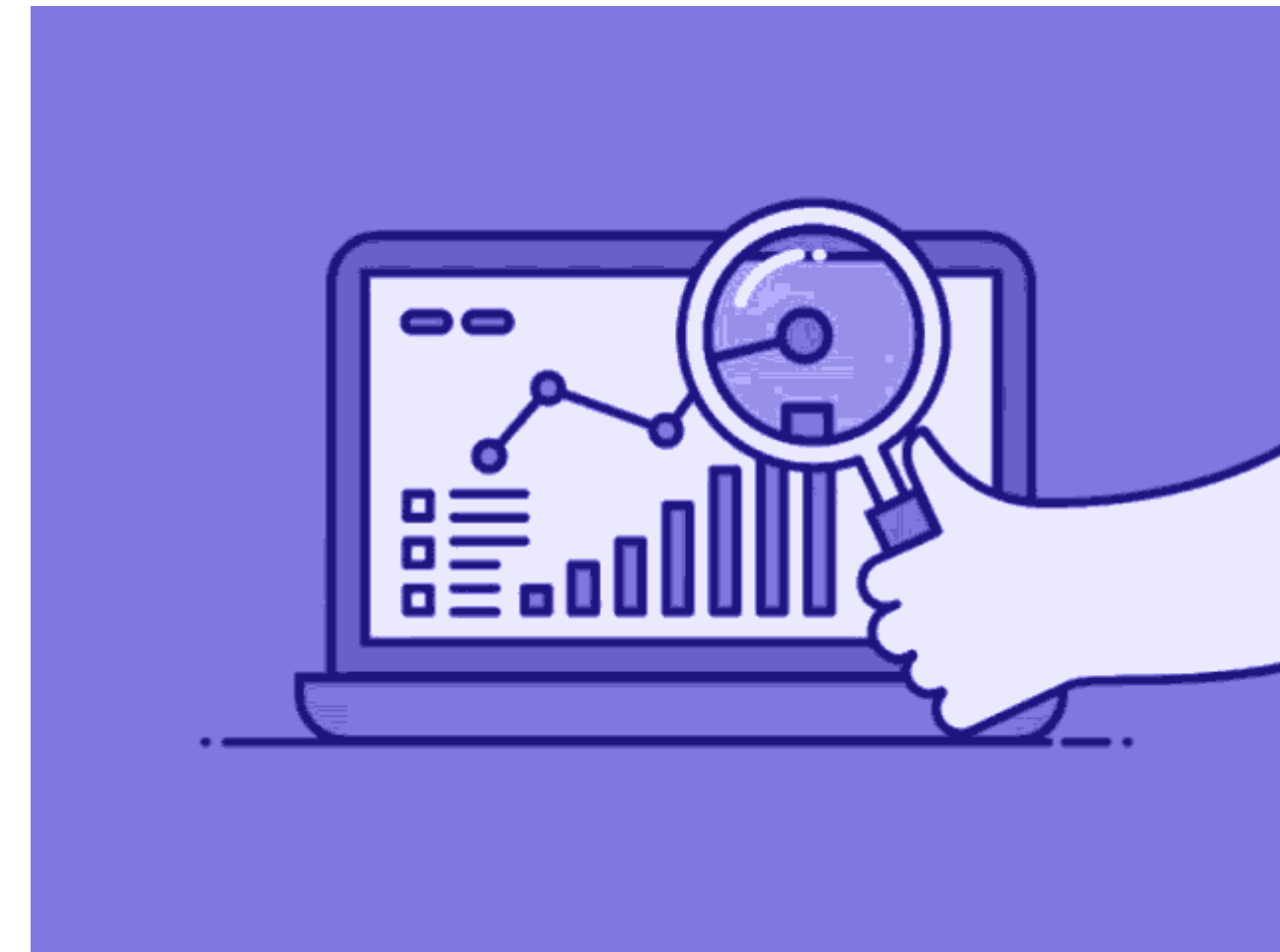
Tantangan dalam pengumpulan data meliputi:

- Data imbalance
- Low-resource language
- Noise tinggi

Data Collection

Ada beberapa jenis data teks diantaranya:

- Raw Corpus** -> Teks tanpa label seperti teks pada Wikipedia
- Anotated Corpus** -> Teks dengan label linguistik atau task-specific seperti POS tagging corpus, Named Entity corpus, Sentiment dataset
- Domain Specific Corpus** -> kumpulan data teks khusus yang disesuaikan untuk memenuhi kebutuhan tertentu dari domain atau aplikasi tertentu seperti medical text, Legal Document



Dalam melakukan pengumpulan data untuk NLP, dapat dilakukan beberapa cara sebagai berikut:

Teknik	Deskripsi	Contoh Sumber
Web Crawling / Scraping	Mengambil teks otomatis dari website menggunakan crawler	Berita online, forum, Wikipedia dump
API-Based Collection	Mengambil data melalui API resmi platform	Twitter API, Reddit API, YouTube API
Public / Benchmark Dataset	Menggunakan dataset yang telah distandardisasi	IMDB Review, Kaggle Dataset
Crowdsourcing	Mengumpulkan atau memberi label data melalui annotator manusia	Mechanical Turk, Appen
Survey / Controlled Experiment	Mengumpulkan data melalui eksperimen atau survei terkontrol	Survei opini, studi psikologi bahasa
Synthetic Data Generation	Membuat data buatan dengan model atau aturan	Data augmentation, back translation

Text cleaning dalam NLP adalah proses awal untuk menghapus **elemen yang tidak relevan, tidak diperlukan**, atau **mengandung noise dari teks mentah** (raw text) sebelum dilakukan analisis linguistik atau pemodelan.

Tujuan Text Cleaning

- Mengurangi noise
- Menstandarkan teks
- Mengurangi dimensi fitur
- Meningkatkan kualitas data
- Menghindari garbage in, garbage out

Sebagai contoh review produk dari media social:

"Saya SUKA banget produk ini!!! 🥰🥰 Cek di <https://shopee.co.id/Pakaian-Pria-cat.11849> #murahbangeet"

Setelah dibersihkan:

saya suka banget produk ini

Garbage In, Garbage Out (GIGO)

Beberapa komponen umum yang dapat diproses dalam text cleaning

1. Case Folding

Proses ini dilakukan untuk mengubah huruf menjadi huruf kecil (tidak kapital)

Contoh : Saya tinggal di Jakarta → saya tinggal di Jakarta

2. Remove HTML Tags

Proses ini dilakukan untuk menghapus html tags yang terambil ketika proses pengumpulan data

contoh : <p>Produk bagus</p> → Produk bagus

3. Remove URL

Menghapus URL yang tidak relevan dalam teks

Contoh: kunjungi <https://shopee.co.id/> → kunjungi

4. Remove Special Characters & Symbols

Proses ini dilakukan untuk menghapus symbol-symbol atau character yang tidak diperlukan. Namun disebagian teks jika dirasa perlu, symbol-symbol atau character tidak dihapus. Contoh mention analysis

5. Remove Numbers

Proses ini dilakukan untuk menghapus angka yang tidak relevan, akan tetapi untuk analisis yang membutuhkan angka, angka tidak boleh dihapus seperti analisis finansial

Contoh : Produk ini 100% bagus → Produk ini bagus

6. Remove Duplicate Text

Menghapus kata-kata yang duplikat, kejadian ini sering terjadi pada data media sosial

7. Remove Extra Whitespace

menghapus spasi ganda atau baris kosong.

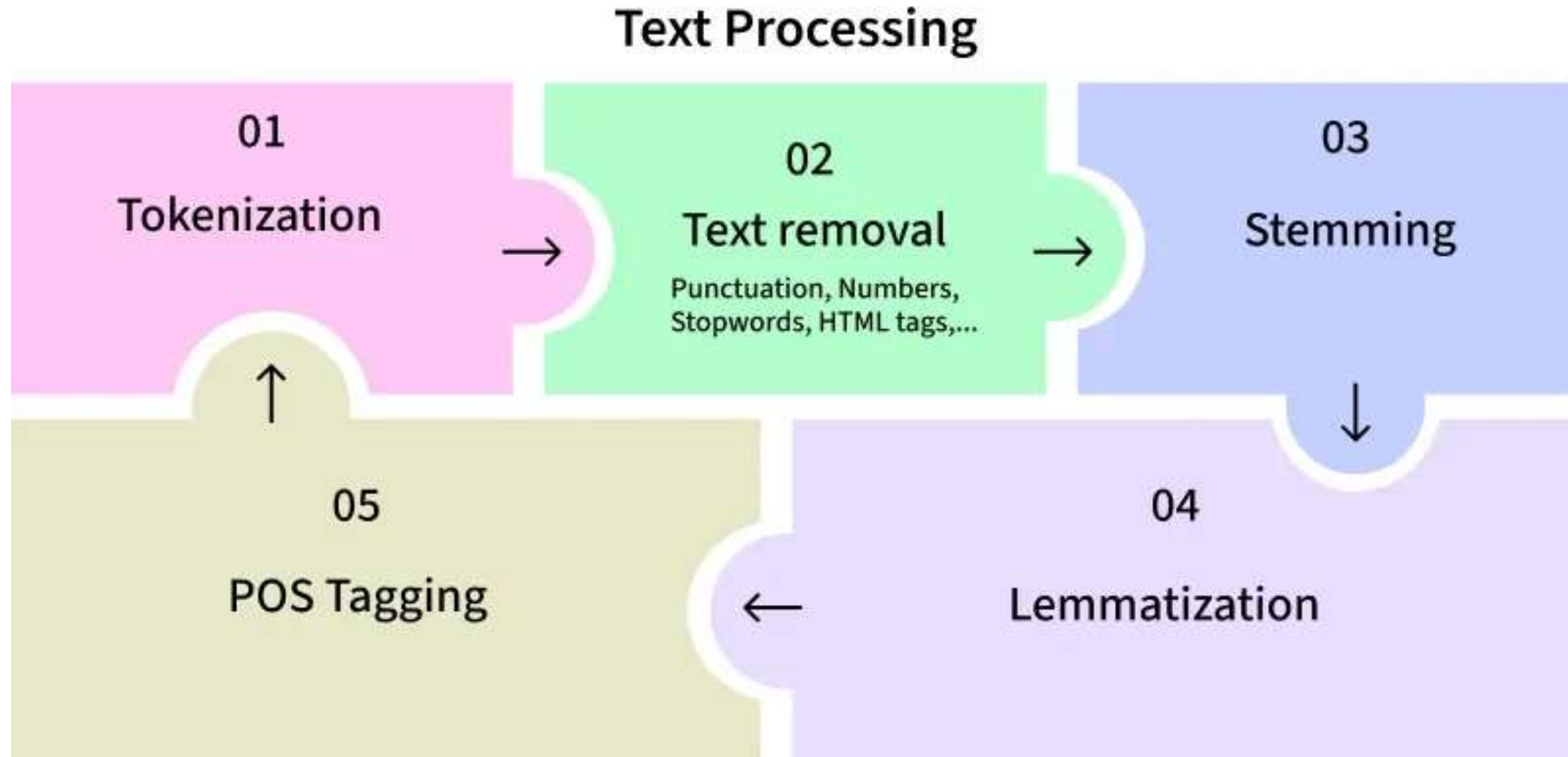
8. Handling Emoji (opsional)

Emoji bisa dihapus dalam beberapa kondisi seperti pada analisis formal, task tidak memerlukan informasi emosional, emoji sangat jarang muncul

Text preprocessing adalah proses transformasi teks mentah menjadi bentuk terstruktur dan terstandar agar dapat diproses secara komputasional oleh model NLP.

Tujuan utama dari melakukan text preprocessing diantaranya:

- Menstandarkan teks
- Mengurangi variasi yang tidak perlu
- Mengurangi sparsity (dimensi terlalu besar)
- Meningkatkan kualitas fitur
- Menyiapkan teks untuk representasi numerik



Normalization adalah proses menyeragamkan berbagai variasi bentuk teks agar memiliki representasi yang konsisten sebelum dilakukan tokenisasi dan modeling.

Beberapa jenis **Normalization** yang dapat dilakukan

1. Case Normalization (Lowercasing)
2. Unicode Normalization
3. Diacritic Removal
4. Text Standardization (Slang Normalization)
5. Handling Repeated Characters
6. Number Normalization
7. Punctuation Normalization
8. Spelling Correction
9. Emoji Normalization
10. Lemmatization sebagai Normalization Lanjutan

Normalization

No	Jenis Normalization	Tujuan	Contoh Sebelum	Contoh Sesudah
1	Case Normalization (Lowercasing)	Menyeragamkan huruf besar/kecil	Saya, saYa, SaYa	saya
2	Unicode Normalization	Menyeragamkan representasi karakter Unicode	café (2 encoding berbeda)	café (format seragam)
3	Diacritic Removal	Menghapus tanda aksen	café	cafe
4	Text Standardization (Slang Normalization)	Mengubah slang/informal menjadi bentuk baku	gk, yg, bgt	tidak, yang, banget
5	Handling Repeated Characters	Mengurangi huruf berulang	baguuuusss	bagus

Normalization

No	Jenis Normalization	Tujuan	Contoh Sebelum	Contoh Sesudah
6	Number Normalization	Menyeragamkan angka	Saya punya 3 buku	Saya punya <NUM> buku
7	Punctuation Normalization	Mengatur atau menghapus tanda baca	Halo!!!	Halo! atau Halo
8	Spelling Correction	Memperbaiki kesalahan ejaan	mantp	mantap
9	Emoji Normalization	Mengubah emoji menjadi bentuk standar		senang atau <EMO_POS>
10	Lemmatization (Normalization Lanjutan)	Mengubah kata ke bentuk dasar	running, ran	run

Tokenization

Tokenization adalah proses memecah teks menjadi unit-unit yang lebih kecil yang disebut **token**, agar dapat diproses oleh sistem NLP. Tanpa tokenization, komputer hanya melihat string panjang tanpa struktur.

Bentuk **token** dapat berupa kata, subword, karakter, atau kalimat

Contoh dari kalimat sebelum dan sesudah tokenization sebagai berikut:

Sebelum tokenization	Sesudah Tokenization
Saya suka rendang	["Saya" , "suka", "rendang"]

Tokenization mempengaruhi beberapa aspek seperti:

- Vocabulary size
- Memory usage
- Training speed
- Generalization
- Context modeling

Jenis Tokenization

1. Sentence Tokenization

- Proses memecah teks menjadi kalimat-kalimat terpisah berdasarkan tanda baca atau aturan linguistik untuk mempertahankan struktur dokumen.
- Unit yang dipecah kalimat.

contoh : “saya membeli bakso, bakso dimakan bersama teman” ->
[“saya membeli bakso”, “bakso dimakan bersama teman”]

2. Word Tokenization

- Memecah kalimat menjadi kata-kata individual. Proses dapat berupa whitespace tokenization, regex-based. Atau statistical tokenizer
- unit yang dipecah adalah kata.

contoh : “saya membeli bakso” -> [“saya” , “makan”, “bakso”]

3. Character Tokenization

- Memecah teks menjadi huruf atau simbol individual untuk analisis pada level paling dasar bahasa.
- Unit yang dipecah adalah karakter tunggal.
- Contoh : “NLP” -> [“N”, “L”, “P”]

4. Subword Tokenization

- Memecah kata menjadi bagian-bagian yang lebih kecil berdasarkan frekuensi kemunculan untuk menangani kata kompleks atau jarang. Biasa digunakan dalam model transformer..
- Memecah sebuah kata menjadi potongan kata.
contoh : "unhappiness" → ["un","happi","ness"]

5. Byte-Level Tokenization

- Memecah teks berdasarkan representasi byte sehingga dapat memproses berbagai bahasa dan simbol tanpa bergantung pada sistem alfabet tertentu.
- Teks dipecah menjadi unit byte.

Stop Word Removal

- **Stop word removal** adalah proses menghapus kata-kata umum yang sering muncul tetapi memiliki sedikit makna penting dalam analisis teks.
- Stop word biasanya berupa kata hubung, kata depan, atau kata bantu.
Contoh stop word Bahasa Indonesia seperti: ke, di, yang, adalah, dan
Contoh stop word Bahasa Inggris seperti: I, am, was, were, the, to
- Tujuan dari adanya stop word removal:
 - Mengurangi jumlah fitur
 - Mengurangi noise
 - Meningkatkan efisiensi komputasi
 - Fokus pada kata yang lebih informatif

Stop Word Removal

- **Contoh stop word removal dalam Bahasa Indonesia:**

Sebelum stop word removal	Sesudah stop word removal
"Rumah itu kebanjiran"	"Rumah kebanjiran"

- **Contoh stop word removal dalam Bahasa Inggris:**

Sebelum stop word removal	Sesudah stop word removal
"I am going to the school"	"going school"

Stemming

- **Stemming** adalah proses mengurangi kata ke bentuk dasarnya (root form) dengan menghapus imbuhan (prefix, suffix) menggunakan aturan heuristik.
- Stemming tidak selalu menghasilkan kata yang valid secara linguistik.
- **Tujuan Stemming**
 - Mengurangi variasi morfologis
 - Mengurangi ukuran vocabulary
 - Mengurangi sparsity fitur
 - Meningkatkan efisiensi model klasik (BoW, TF-IDF)

Kata Asli	Hasil Stemming
running	run
played	play
studies	studi
berlari	lari
makanan	makan
dimainkan	main

Lemmatization

- **Lemmatization** adalah proses mengubah kata ke bentuk dasarnya (lemma) berdasarkan analisis morfologi dan kamus linguistik.
- Berbeda dengan stemming, lemmatization mempertimbangkan konteks dan part-of-speech (POS).
- Lemmatization menyatukan berbagai bentuk kata ke bentuk dasar yang benar secara linguistik agar model memahami makna secara lebih konsisten dan akurat.

Kata Asli	Hasil Lemmatization
running	run
better	good
was	be
studies	study
meminjam	pinjam
dimakan	makan
tertibur	tidur

Stemming vs Lemmatization

Aspek	Stemming	Lemmatization
Pendekatan	Heuristik (rule-based)	Linguistik + kamus
Akurasi	Lebih rendah	Lebih tinggi
Kecepatan	Cepat	Lebih lambat
Hasil valid linguistik	Tidak selalu	Ya
Butuh POS tagging	Tidak	Ya

Contoh Kalimat:

The children are running faster than the adults

Setelah Stemming:

the children are run faster than the adult

Setelah Lemmatization:

the children are run faster than the adult

Part of Speech (POS)

Part-of-Speech (PoS) Tagging adalah tugas fundamental dalam Natural Language Processing (NLP) yang bertujuan untuk memberikan label kategori gramatikal pada setiap kata dalam sebuah kalimat. Kategori tersebut meliputi:

- Noun (kata benda)
- Verb (kata kerja)
- Adjective (kata sifat)
- Adverb (kata keterangan)
- Pronoun (kata ganti)
- Preposition (kata depan)
- Conjunction (kata hubung)

Beberapa tipe PoS Tagging seperti:

1. Rule-Based

Menggunakan aturan tata bahasa dan kamus untuk menentukan kategori kata.

2. Statistical (Probabilistic)

Menggunakan probabilitas dari data berlabel untuk memilih tag yang paling mungkin (misalnya HMM, CRF).

3. Neural / Deep Learning

Menggunakan model jaringan saraf seperti BERT untuk mempelajari pola secara otomatis dari data.

Keuntungan dari POS Tagging:

- **Text Simplification:** Penandaan POS dapat memecah kalimat kompleks menjadi bagian-bagian yang lebih sederhana, sehingga lebih mudah dipahami.
- **Peningkatan Pencarian:** POS Tagging meningkatkan pengambilan informasi dengan mengkategorikan kata-kata, sehingga lebih mudah untuk mengindeks dan mencari teks.
- **Named Entity Recognition (NER):** POS Tagging membantu mengidentifikasi nama, tempat, dan organisasi dengan mengenali peran kata dalam sebuah kalimat.

Tantangan dalam POS Tagging

1. Ambiguitas:

Sebuah kata dapat memiliki lebih dari satu makna tergantung pada konteksnya, sehingga menyulitkan proses pemberian tag.

2. Ekspresi Idiomatik:

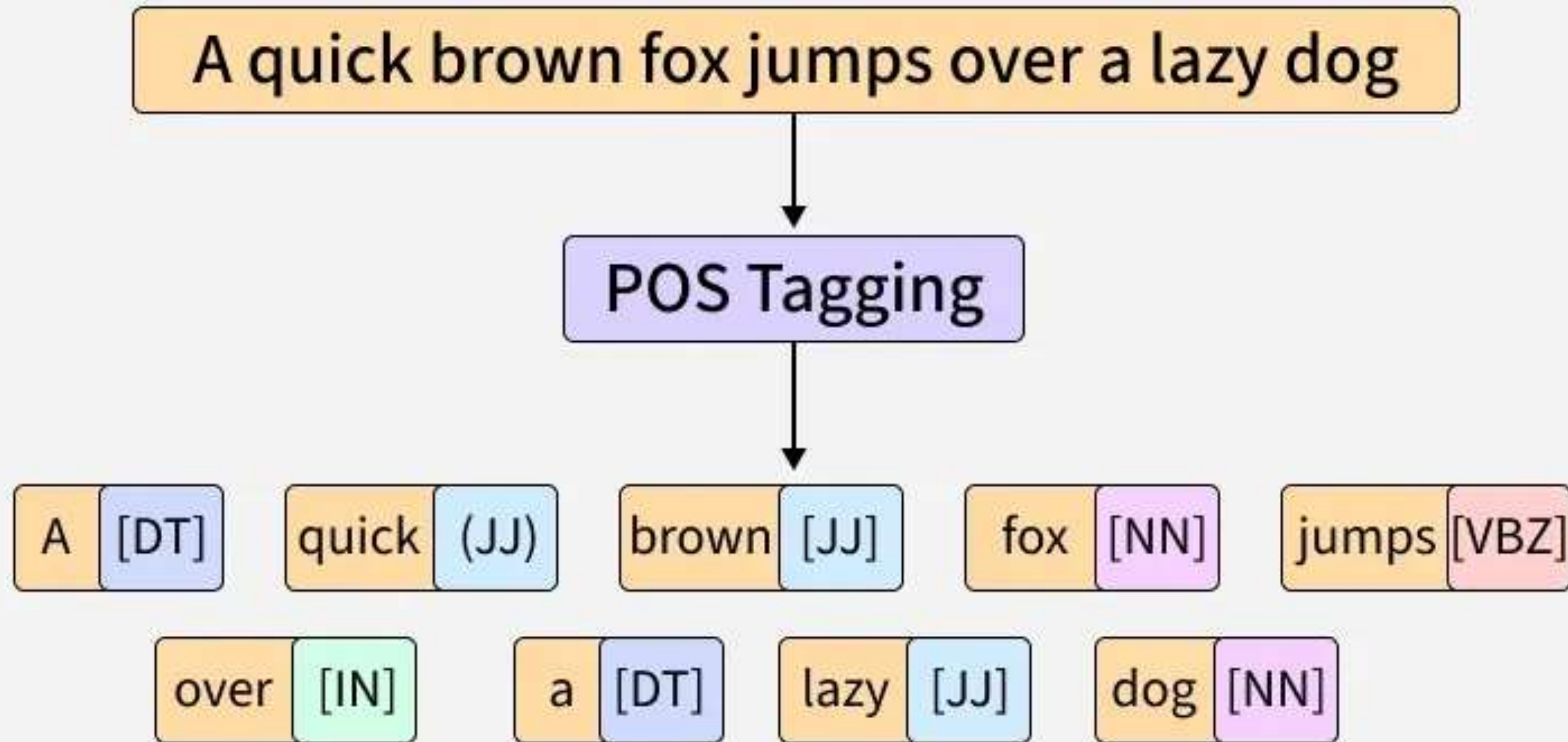
Bahasa informal dan ungkapan idiomatik sering kali sulit diberi tag secara tepat karena maknanya tidak selalu sesuai dengan arti kata secara literal.

3. Out-of-Vocabulary (OOV):

Kata-kata yang belum pernah muncul sebelumnya dalam data pelatihan dapat menyebabkan kesalahan dalam pemberian tag.

4. Ketergantungan Domain:

Model POS tagging mungkin tidak bekerja dengan baik jika digunakan pada domain atau jenis teks yang berbeda dari data yang digunakan saat pelatihan.



Upcoming Topics

- Feature Engineering
- Modeling
- Evaluation
- Deployment
- Monitoring

These topics will be discussed in the next session!

Thank you guys!